

Two-Dimensional Arrays in Java

Introduction

A **Two-Dimensional (2D) Array** in Java is an array of arrays used to store data in the form of rows and columns. It is useful for representing tables, matrices, spreadsheets, game boards, seating arrangements, and other tabular data.

A one-dimensional array stores elements in a single row, whereas a two-dimensional array stores elements in multiple rows and columns.

Example:

Row/Column	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

Here,

- Number of rows = 3
 - Number of columns = 3
 - Total elements = 9
-

Declaration of a 2D Array

Syntax

```
dataType[][] arrayName;
```

or

```
dataType arrayName[][];
```

Example

```
int[][] marks;
```

This declaration creates a reference variable for a two-dimensional integer array.

Creating a 2D Array

A two-dimensional array is created using the **new** keyword.

Syntax

```
arrayName = new dataType[rows][columns];
```

Example

```
int[][] marks = new int[3][4];
```

This creates a matrix with

- 3 rows
- 4 columns

Total elements = $3 \times 4 = 12$

Declaring and Creating Together

```
int[][] numbers = new int[2][3];
```

This creates the following structure:

Row 0 → [0][0] [0][1] [0][2]

Row 1 → [1][0] [1][1] [1][2]

Initially, all integer values are **0**.

Initialization of a 2D Array

Method 1: Using Values

```
int[][] a = {  
    {10,20,30},  
    {40,50,60},  
    {70,80,90}  
};
```

Method 2: Assigning Individual Elements

```
int[][] a = new int[2][2];
```

```
a[0][0] = 5;
```

```
a[0][1] = 10;
```

```
a[1][0] = 15;
```

```
a[1][1] = 20;
```

Accessing Elements

Each element is identified using

`arrayName[row][column]`

Example

```
System.out.println(a[1][2]);
```

Output

60

because

Row 1

Column 2

contains 60.

Reading Elements from the User

```
Scanner sc = new Scanner(System.in);
```

```
int[][] a = new int[3][3];
```

```
for(int i=0;i<3;i++)
{
    for(int j=0;j<3;j++)
    {
        a[i][j]=sc.nextInt();
    }
}
```

The outer loop represents rows.

The inner loop represents columns.

Displaying a 2D Array

```
for(int i=0;i<3;i++)
{
    for(int j=0;j<3;j++)
    {
```

```
        System.out.print(a[i][j]+" ");  
    }  
    System.out.println();  
}
```

Output

1 2 3

4 5 6

7 8 9

Memory Representation

Suppose

```
int[][] a = {  
    {1,2,3},  
    {4,5,6}  
};
```

Memory representation

Row 0

1 2 3

Row 1

4 5 6

Indices

$a[0][0] = 1$

$a[0][1] = 2$

$a[0][2] = 3$

$a[1][0] = 4$

`a[1][1] = 5`

`a[1][2] = 6`

Length of a 2D Array

The number of rows

`a.length`

The number of columns

`a[0].length`

Example

```
System.out.println("Rows = " + a.length);
```

```
System.out.println("Columns = " + a[0].length);
```

Output

Rows = 2

Columns = 3

Traversing a 2D Array

```
for(int i=0;i<a.length;i++)  
{  
    for(int j=0;j<a[i].length;j++)  
    {  
        System.out.print(a[i][j]+" ");  
    }  
    System.out.println();  
}
```

Using `a.length` and `a[i].length` is preferred because it works for arrays of different sizes.

Enhanced for Loop

Java also supports the enhanced for loop.

Example

```
int[][] a={
    {1,2,3},
    {4,5,6}
};

for(int[] row:a)
{
    for(int value:row)
    {
        System.out.print(value+" ");
    }
    System.out.println();
}
```

Output

1 2 3

4 5 6

Common Operations on 2D Arrays

1. Matrix Addition

$$C[i][j] = A[i][j] + B[i][j]$$

2. Matrix Subtraction

$$C[i][j] = A[i][j] - B[i][j]$$

3. Matrix Multiplication

$$C[i][j] = \sum A[i][k] \times B[k][j]$$

This operation requires three nested loops.

4. Transpose of a Matrix

Rows become columns.

Example

Original

1 2 3

4 5 6

Transpose

1 4

2 5

3 6

5. Sum of All Elements

$\text{sum} = \text{sum} + a[i][j];$

6. Row Sum

Calculate the sum of every row separately.

7. Column Sum

Calculate the sum of every column separately.

8. Largest and Smallest Element

Compare every element with the current maximum or minimum value.

9. Searching an Element

Compare every element with the search key.

10. Diagonal Elements

Primary diagonal

`a[i][i]`

Secondary diagonal

`a[i][n-1-i]`

Example

1 2 3

4 5 6

7 8 9

Primary diagonal

1 5 9

Secondary diagonal

3 5 7

Jagged Arrays

Java allows rows to have different numbers of columns. Such arrays are called **jagged arrays** or **ragged arrays**.

Example

```
int[][] a = new int[3][];
```

```
a[0] = new int[2];
```

```
a[1] = new int[4];
```

```
a[2] = new int[3];
```

Structure

Row 0 → 2 elements

Row 1 → 4 elements

Row 2 → 3 elements

Advantages of 2D Arrays

- Store data in tabular form.
- Easy representation of matrices.
- Suitable for scientific and mathematical computations.
- Simple indexing using row and column.
- Efficient for processing structured data.
- Widely used in games, image processing, and database applications.

Disadvantages

- Fixed size once created.
- Memory consumption increases with array size.
- Insertion and deletion are difficult.
- Large arrays may waste memory if many elements are unused.

Applications of 2D Arrays

- Matrix operations
- Image processing
- Computer graphics
- Game boards (Chess, Sudoku, Tic-Tac-Toe)
- Student mark lists
- Attendance registers
- Seating arrangements
- Spreadsheet software
- Scientific computing
- Database tables

Difference Between 1D and 2D Arrays

One-Dimensional Array	Two-Dimensional Array
-----------------------	-----------------------

Stores data in a single list	Stores data in rows and columns
------------------------------	---------------------------------

Uses one index

Uses two indices

Syntax: `a[i]`

Syntax: `a[i][j]`

Single loop for traversal

Nested loops for traversal

Represents a list

Represents a matrix or table